

Requisitos Funcionales

Información para conservar:

- **Flota:** Vehículos registrados (placa, marca, modelo, tarifa, estado, días acumulados para mantenimiento) y características específicas (pasajeros, transmisión, cilindraje, capacidad, piloto).
- **Clientes:** Identificador único, nombre, licencias, tipo (individual o corporativo), historial de alquileres y conteo de alquileres activos.
- **Alquileres:** Registro correlativo, vehículo alquilado, cliente, días, subtotal, descuento y total.
- **Sistema:** Ingresos acumulados de manera global y por categoría y descuentos totales aplicados.

Operaciones:

- Registrar vehículos y clientes.
- Consultar flota, clientes y reportes diarios.
- Cotizar alquileres (cálculo de costos y validación).
- Confirmar o cancelar alquileres.
- Registrar devoluciones y fin de mantenimiento.

Condiciones (Validaciones):

- No duplicidad de placas ni identificadores de clientes.
- Validación de tarifas, días y medidas como enteros o decimales positivos mayores a cero.
- Comprobación de estado " Disponible " para alquilar.
- Validación de licencias según categoría y validación de límite de alquileres activos (1 para individual, 3 para corporativo).
- Umbral de días acumulados para enviar a mantenimiento.

Tablas de Clases, Atributos y Métodos

Clase	Responsabilidad
Main	Punto de entrada. Instancia únicamente el Controlador y llama a su método de inicio para arrancar la ejecución.
VistaRentaMovil	Maneja la interfaz de consola, captura entradas (Scanners), formatea la salida de texto y atrapa excepciones de formato.
ControladorRentaMovil	Orquesta la lógica del negocio. Almacena las colecciones polimórficas de la flota y clientes. Comunica la Vista con el Modelo sin exponer la lógica interna.

Atributos y Métodos de la Vista y Controlador:

Clase	Atributo	Tipo de Dato	Visibilidad	Propósito / Valor Inicial
VistaRentaMovil	scanner	Scanner	private	Leer entrada del usuario.
VistaRentaMovil	controlador	ControladorRentaMovil	private	Referencia para solicitar operaciones.
ControladorRentaMovil	vehiculos	List<Vehiculo>	private	Colección polimórfica. Inicializa con al menos 2 por categoría.
ControladorRentaMovil	clientes	List<Cliente>	private	Colección polimórfica. Inicializa con al menos 2 por tipo.
ControladorRentaMovil	alquileres	List<Alquiler>	private	Historial de alquileres confirmados.
ControladorRentaMovil	correlativoActual	int	private	Genera números correlativos. Inicia en 1.
ControladorRentaMovil	ingresosTotales	double	private	Acumulador global de dinero. Inicia en 0.0.
ControladorRentaMovil	descuentosTotales	double	private	Acumulador global de descuentos. Inicia en 0.0.

Clase	Método	Parámetros	Tipo de Retorno	Visibilidad	Propósito
Main	main	String[] args	void	public static	Invoca la inicialización del sistema.
VistaRentaMovil	mostrarMenu	Ninguno	void	public	Imprime el menú principal y captura la opción.
VistaRentaMovil	leerTexto	String mensaje	String	private	Imprime mensaje y lee un texto.
VistaRentaMovil	leerEntero	String mensaje	int	private	Imprime mensaje, lee un entero, atrapa InputMismatchException.
VistaRentaMovil	leerDouble	String mensaje	double	private	Imprime mensaje, lee un decimal, atrapa error formato.
ControladorRentaMovil	iniciarSistema	Ninguno	void	public	Carga datos iniciales y llama a vista.mostrarMenu().
ControladorRentaMovil	registrarVehiculo	Vehiculo v	void	public	Valida placa única y agrega a la lista.
ControladorRentaMovil	registrarCliente	Cliente c	void	public	Valida ID único y agrega a la lista.
ControladorRentaMovil	buscarVehiculo	String placa	Vehiculo	public	Devuelve el vehículo o null si no existe.
ControladorRentaMovil	buscarCliente	String id	Cliente	public	Devuelve el cliente o null si no existe.
ControladorRentaMovil	cotizarAlquiler	String placa, String id, int dias	String	public	Calcula y retorna el detalle sin guardar cambios.
ControladorRentaMovil	confirmarAlquiler	String placa, String id, int dias	boolean	public	Ejecuta la lógica del alquiler, actualiza estados e ingresos.
ControladorRentaMovil	registrarDevolucion	String placa	boolean	public	Cambia estado del vehículo y evalúa umbral de mantenimiento.
ControladorRentaMovil	registrarFinMantenimiento	String placa	boolean	public	Cambia estado a DISPONIBLE y reinicia días acumulados.
ControladorRentaMovil	generarReportes	Ninguno	String	public	Retorna los cálculos de métricas solicitadas.

Modelo: Vehículos (Herencia)

Clase	Responsabilidad
Vehiculo (Abstracta)	Define los atributos comunes, encapsula el estado y define métodos polimórficos obligatorios. No debe instanciarse porque todo vehículo debe pertenecer a una categoría específica para sus reglas de cobro.
Automovil, Motocicleta, CamionetaCarga, Microbus	Subclases que sobrescriben los métodos de la superclase para aplicar sus reglas específicas.
EstadoVehiculo (Enum)	Define DISPONIBLE, ALQUILADO, EN_MANTENIMIENTO.

Atributos de Vehículos:

Clase	Atributo	Tipo de Dato	Visibilidad	Propósito / Valor Inicial
Vehiculo	placa	String	private	Identificador. Recibido en constructor.
Vehiculo	marca	String	private	Recibido en constructor.
Vehiculo	modelo	String	private	Recibido en constructor.
Vehiculo	tarifaDiaria	double	private	Recibido en constructor.
Vehiculo	estado	EstadoVehiculo	private	Inicia siempre como EstadoVehiculo.DISPONIBLE.
Vehiculo	diasAcumulados	int	private	Acumulador para mantenimiento. Inicia en 0.
Automovil	pasajeros	int	private	Recibido en constructor.
Automovil	esAutomatico	boolean	private	Recibido en constructor.
Motocicleta	cilindraje	int	private	Recibido en constructor.
CamionetaCarga	capacidadToneladas	double	private	Recibido en constructor.
Microbus	pasajeros	int	private	Recibido en constructor.
Microbus	conPiloto	boolean	private	Fijo, recibido en constructor.

Métodos de Vehículos:

Clase	Método	Parámetros	Tipo de Retorno	Visibilidad	Propósito
Vehiculo	getPlaca	Ninguno	String	public	Retorna la placa.
Vehiculo	setPlaca	String placa	void	public	Asigna la placa.
Vehiculo	getMarca	Ninguno	String	public	Retorna la marca.
Vehiculo	setMarca	String marca	void	public	Asigna la marca.
Vehiculo	getModelo	Ninguno	String	public	Retorna el modelo.
Vehiculo	setModelo	String modelo	void	public	Asigna el modelo.
Vehiculo	getTarifaDiaria	Ninguno	double	public	Retorna la tarifa diaria.
Vehiculo	setTarifaDiaria	double tarifa	void	public	Asigna la tarifa diaria.
Vehiculo	getEstado	Ninguno	EstadoVehiculo	public	Retorna el estado actual.
Vehiculo	setEstado	EstadoVehiculo estado	void	public	Asigna el estado.
Vehiculo	getDiasAcumulados	Ninguno	int	public	Retorna días de uso.
Vehiculo	setDiasAcumulados	int dias	void	public	Asigna días de uso.
Vehiculo	sumarDiasMantenimiento	int dias	void	public	Suma días tras alquiler.
Vehiculo	calcularRecargo (Abstract)	int dias	double	public	Determina el recargo según la subclase.
Vehiculo	validarLicencia (Abstract)	List<TipoLicencia> licencias	boolean	public	Verifica si las licencias del cliente son aptas.
Vehiculo	getUmbralMantenimiento (Abstract)	Ninguno	int	public	Retorna el límite antes del taller.
Vehiculo	getDescripcion (Abstract)	Ninguno	String	public	Retorna las características específicas (polimorfismo).
Automovil	getPasajeros	Ninguno	int	public	Retorna cantidad de pasajeros.
Automovil	setPasajeros	int pasajeros	void	public	Asigna pasajeros.

Automovil	isEsAutomatico	Ninguno	boolean	public	Retorna si es automático.
Automovil	setEsAutomatico	boolean esAutomatico	void	public	Asigna transmisión.
Automovil	calcularRecargo (Override)	int dias	double	public	Retorna Q50 * días si es automático, o si no.
Automovil	validarLicencia (Override)	List<TipoLicencia> licencias	boolean	public	Retorna true si hay licencia A, B o C.
Automovil	getUmbralMantenimiento (Override)	Ninguno	int	public	Retorna 30.
Automovil	getDescripcion (Override)	Ninguno	String	public	Concatena datos + pasajeros + transmisión.
Motocicleta	getCilindraje	Ninguno	int	public	Retorna cilindraje.
Motocicleta	setCilindraje	int cilindraje	void	public	Asigna cilindraje.
Motocicleta	calcularRecargo (Override)	int dias	double	public	Retorna Q75 si cilindraje > 250, o si no.
Motocicleta	validarLicencia (Override)	List<TipoLicencia> licencias	boolean	public	Retorna true si hay licencia M.
Motocicleta	getUmbralMantenimiento (Override)	Ninguno	int	public	Retorna 20.
Motocicleta	getDescripcion (Override)	Ninguno	String	public	Concatena datos + cilindraje.
CamionetaCarga	getCapacidadToneladas	Ninguno	double	public	Retorna toneladas.
CamionetaCarga	setCapacidadToneladas	double toneladas	void	public	Asigna toneladas.
CamionetaCarga	calcularRecargo (Override)	int dias	double	public	Retorna Q100 * toneladas * días.
CamionetaCarga	validarLicencia (Override)	List<TipoLicencia> licencias	boolean	public	Retorna true si hay licencia A o B.
CamionetaCarga	getUmbralMantenimiento (Override)	Ninguno	int	public	Retorna 15.
CamionetaCarga	getDescripcion (Override)	Ninguno	String	public	Concatena datos + toneladas.
Microbus	getPasajeros	Ninguno	int	public	Retorna pasajeros.
Microbus	setPasajeros	int pasajeros	void	public	Asigna pasajeros.
Microbus	isConPiloto	Ninguno	boolean	public	Retorna si tiene piloto.
Microbus	setConPiloto	boolean conPiloto	void	public	Asigna piloto.
Microbus	calcularRecargo (Override)	int dias	double	public	Retorna Q250 * días si tiene piloto, o si no.
Microbus	validarLicencia (Override)	List<TipoLicencia> licencias	boolean	public	True sin evaluar si tiene piloto; o evalúa A/B si no.
Microbus	getUmbralMantenimiento (Override)	Ninguno	int	public	Retorna 25.
Microbus	getDescripcion (Override)	Ninguno	String	public	Concatena datos + pasajeros + piloto.

Modelo: Clientes (Herencia)

Clase	Responsabilidad
Cliente (Abstracta)	Maneja el identificador, nombre, licencias, y mantiene el estado de alquileres. Abstracta porque las reglas de descuento y límites dependen de si es individual o corporativo.
ClienteIndividual, ClienteCorporativo	Aplican las reglas de negocio específicas para sus alquileres y descuentos.
TipoLicencia (Enum)	Define A, B, C, M.

Atributos de Clientes:

Clase	Atributo	Tipo de Dato	Visibilidad	Propósito / Valor Inicial
Cliente	identificador	String	private	DPI o NIT, recibido en constructor.
Cliente	nombre	String	private	Recibido en constructor.
Cliente	licencias	List<TipoLicencia>	private	Lista de licencias presentadas.
Cliente	alquileresActivos	int	private	Conteo en curso. Inicia en 0.
Cliente	historialAlquileres	List<Alquiler>	private	Alquileres de este cliente.
ClienteIndividual	alquileresConfirmados	int	private	Conteo histórico para evaluar descuento. Inicia en 0.
ClienteCorporativo	contacto	String	private	Nombre de contacto. Recibido en constructor.

Métodos de Clientes:

Clase	Método	Parámetros	Tipo de Retorno	Visibilidad	Propósito
Cliente	getIdentificador	Ninguno	String	public	Retorna DPI o NIT.
Cliente	setIdentificador	String id	void	public	Asigna identificador.
Cliente	getNombre	Ninguno	String	public	Retorna nombre.
Cliente	setNombre	String nombre	void	public	Asigna nombre.
Cliente	getLicencias	Ninguno	List<TipoLicencia>	public	Retorna lista de licencias.
Cliente	setLicencias	List<TipoLicencia> licencias	void	public	Asigna licencias.
Cliente	getAlquileresActivos	Ninguno	int	public	Retorna alquileres activos.
Cliente	setAlquileresActivos	int alquileres	void	public	Asigna alquileres activos.
Cliente	getHistorialAlquileres	Ninguno	List<Alquiler>	public	Retorna historial del cliente.
Cliente	agregarAlquilerHistorial	Alquiler a	void	public	Añade el alquiler a su lista interna.
Cliente	calcularDescuento (Abstract)	double subtotal	double	public	Determina el descuento según el tipo de cliente.
Cliente	puedeAlquilarMas (Abstract)	Ninguno	boolean	public	Valida si no superó su límite de activos.
Cliente	registrarAlquilerConfirmado (Abstract)	Ninguno	void	public	Incrementa contadores históricos si es necesario.
ClienteIndividual	getAlquileresConfirmados	Ninguno	int	public	Retorna confirmados.
ClienteIndividual	setAlquileresConfirmados	int confirmados	void	public	Asigna confirmados.
ClienteIndividual	calcularDescuento (Override)	double subtotal	double	public	Retorna 5% del subtotal si alquileresConfirmados >= 3, si no 0.0.
ClienteIndividual	puedeAlquilarMas (Override)	Ninguno	boolean	public	Retorna true si alquileresActivos < 1.
ClienteIndividual	registrarAlquilerConfirmado	Ninguno	void	public	alquileresConfirmados++.
ClienteCorporativo	getContacto	Ninguno	String	public	Retorna contacto.
ClienteCorporativo	setContacto	String contacto	void	public	Asigna contacto.
ClienteCorporativo	calcularDescuento (Override)	double subtotal	double	public	Retorna siempre 10% del subtotal.
ClienteCorporativo	puedeAlquilarMas (Override)	Ninguno	boolean	public	Retorna true si alquileresActivos < 3.
ClienteCorporativo	registrarAlquilerConfirmado	Ninguno	void	public	Vacío, ya que los corporativos no

					dependen de este historial.
--	--	--	--	--	-----------------------------

Modelo: Alquiler

Atributos de Alquiler:

Clase	Atributo	Tipo de Dato	Visibilidad	Propósito / Valor Inicial
Alquiler	correlativo	int	private	Identificador único del registro.
Alquiler	cliente	Cliente	private	Referencia al cliente que alquila.
Alquiler	vehiculo	Vehiculo	private	Referencia al vehículo alquilado.
Alquiler	dias	int	private	Duración. Recibido en constructor.
Alquiler	subtotal	double	private	Calculado al crear (Tarifa * días + Recargo).
Alquiler	descuento	double	private	Calculado al crear.
Alquiler	total	double	private	Calculado al crear (subtotal - descuento).

Métodos de Alquiler:

Clase	Método	Parámetros	Tipo de Retorno	Visibilidad	Propósito
Alquiler	getCorrelativo	Ninguno	int	public	Retorna el correlativo.
Alquiler	getCliente	Ninguno	Cliente	public	Retorna el cliente asociado.
Alquiler	getVehiculo	Ninguno	Vehiculo	public	Retorna el vehículo.
Alquiler	getDias	Ninguno	int	public	Retorna los días.
Alquiler	getSubtotal	Ninguno	double	public	Retorna el subtotal.
Alquiler	getDescuento	Ninguno	double	public	Retorna el descuento.
Alquiler	getTotal	Ninguno	double	public	Retorna el total.

Justificación de Jerarquías y Polimorfismo

- **Jerarquías de Clases Abstractas** (Vehículo y Cliente): Las superclases capturan los elementos comunes, como placa o identificador de las colecciones, evitando código repetido. No deben instanciarse porque un "Vehículo" genérico carece de un umbral de mantenimiento o cálculo de recargo específico; igualmente, un "Cliente" genérico no posee una regla concreta de descuento.
- **Comportamientos Polimórficos**: En el caso de los vehículos, calcularRecargo(int dias), validarLicencia(List<TipoLicencia> licencias), getUmbralMantenimiento() y getDescripcion() son métodos polimórficos. Su ejecución la decide el objeto instanciado, el tipo real. Esto permite a ControladorRentaMovil calcular un subtotal con vehiculo.getTarifaDiaria() * dias + vehiculo.calcularRecargo(dias) sin necesidad de usar instanceof para preguntar si el objeto es motocicleta o microbús.

Decisión sobre los Datos Repetidos

Tanto Automovil como Microbus requieren el atributo pasajeros. Este dato se ubica en las subclases específicas y no en la superclase Vehiculo. Si se colocara en la superclase, clases como Motocicleta o CamionetaCarga heredarían un dato que no es relevante para el negocio en su dominio, rompiendo el principio de cohesión y el diseño orientado a objetos.

Recorrido Completo: Confirmación de un Alquiler

1. La **Vista** (VistaRentaMovil) solicita placa, identificación y días mediante teclado y llama a controlador.cotizarAlquiler(...).
2. El **Controlador** (ControladorRentaMovil) busca el vehículo (de la colección común Vehiculo) y el cliente (de la colección común Cliente).
3. El **Controlador** evalúa vehiculo.getEstado(). Si no es DISPONIBLE, rechaza la acción.

4. El **Controlador** invoca polimórficamente `vehiculo.validarLicencia(cliente.getLicencias())` y `cliente.puedeAlquilarMas()`. La subclase evalúa sus propias reglas, si alguna retorna falso, se rechaza la cotización con el motivo.
5. Se calcula el cobro: tarifa multiplicada por días, más el resultado de `vehiculo.calcularRecargo(dias)`. Al resultado se le aplica `cliente.calcularDescuento(subtotal)`.
6. La **Vista** imprime el detalle y pregunta si el cliente acepta.
7. Si acepta, la Vista llama a `controlador.confirmarAlquiler(...)`.
8. El **Controlador** instancia un nuevo Alquiler, modifica estado a ALQUILADO, incrementa los activos del cliente, invoca `cliente.registrarAlquilerConfirmado()`, acumula los ingresos en la variable global del controlador y agrega el alquiler al historial.

Evaluación de la Extensión

Para incorporar vehículos eléctricos y clientes gubernamentales en el futuro, gracias al diseño polimórfico:

- **Clases a crear:** `VehiculoElectrico`, heredando de `Vehiculo` y `ClienteGubernamental`, heredando de `Cliente`. Estas implementarán sus propias versiones de `calcularRecargo`, `getUmbralMantenimiento`, `calcularDescuento`.
- **Partes del sistema que NO cambian:** La clase `ControladorRentaMovil` responsable de registrar alquileres o calcular totales no requiere de estructuras condicionales (`if/switch` ni `instanceof`) para lidiar con los nuevos tipos, su lógica principal permanece inalterada, así como la clase `Alquiler`. Solo la `VistaRentaMovil` requerirá ajustes menores para habilitar la opción de registrar la creación de estos nuevos objetos.